

Agentic FacilityOps AI Platform

Predictive Maintenance Module

Technical Project Documentation

Version 1.0 | Predictive Maintenance Module

Date: 15 August 2026

Author: G Vinay Kumar | Agentic AI Engineering

Table of Contents

1. Project Overview & Objectives	pg. 3
2. Problem Statement & Business Impact	pg. 3
3. Technology Stack	pg. 4
4. System Architecture	pg. 5
5. Database Design	pg. 6
6. Backend — FastAPI Application	pg. 7
7. AI Maintenance Agent — Health & Predictions	pg. 8
8. Analytics & Scoring Engine	pg. 9
9. REST API Endpoints	pg. 10
10. Frontend — React Dashboard	pg. 11
11. Dashboard Pages & Features	pg. 12
12. Data Simulation & Microsoft Azure Dataset	pg. 14
13. Report Export Engine	pg. 14
14. Licenses & Open Source	pg. 15
15. How to Run the Application	pg. 15
16. Future Roadmap	pg. 16

1. Project Overview & Objectives

Agentic FacilityOps AI Platform is a production-grade, enterprise-level AI-powered Predictive Maintenance system for commercial buildings such as IT Parks, Hospitals, Universities, Shopping Malls, and Factories. The platform leverages the real-world **Microsoft Azure Predictive Maintenance** dataset to simulate IoT telemetry (vibration, pressure, temperature) across 100 machines. The AI Agent analyses the usage, predicts component failures, generates health scores, and visualises everything through a professional dashboard.

1.1 Core Objectives

Objective	Description
Real-Time Telemetry	Collect and display hourly voltage, rotation, pressure, and vibration from IoT sensors.
Health Scoring	Calculate a dynamic 0-100 health score for every piece of equipment based on recent telemetry anomalies.
Failure Prediction	Automatically detect warning signs of equipment failure before they happen (e.g., compressor anomalies).
Maintenance Scheduling	Track predictive, preventive, and corrective maintenance tasks assigned to engineers.
Alerts & Interventions	Generate AI-driven alerts for impending breakdowns and suggest repair recommendations.
Multi-tenant Dashboard	A fully reactive SPA with glassmorphism design for managing equipment across multiple facilities.

2. Problem Statement & Business Impact

Unexpected equipment failure (HVAC units, generators, pumps) is one of the highest costs in facility management. Traditional 'preventive' maintenance replaces parts too early, while 'reactive' maintenance waits for costly breakdowns.

Challenge	Traditional Approach	Our Solution
Asset Visibility	Clipboard inspections	Live digital twin dashboard
Maintenance Timing	Calendar-based (Fixed schedule)	Condition-based (AI Predictive)
Downtime Prevention	Wait for breakdown	Detect degradation days in advance
Lifecycle Tracking	Spreadsheets	Automated health scores & runtime
Task Management	Paper work-orders	Digital schedules & AI alerts

Business Impact: Moving from reactive to AI predictive maintenance can reduce unplanned downtime by 30-50% and increase equipment lifespan by 20%, saving tens of thousands of dollars annually.

3. Technology Stack

3.1 Backend

Technology	Version	License	Purpose
Python	3.11+	PSF (Free)	Core programming language
FastAPI	>=0.110	MIT	REST API web framework with auto Swagger docs
SQLAlchemy ORM	>=2.0	MIT	Database ORM — write Python instead of SQL
Pydantic	>=2.6	MIT	Automatic request/response data validation
Pandas	>=2.2	BSD	Vectorised analytics and CSV data processing
NumPy	>=1.26	BSD	Statistical generation for Azure PdM Dataset
ReportLab	>=4.1	BSD	PDF audit report generation
OpenPyXL	>=3.1	MIT	Excel (.xlsx) report generation
Uvicorn	>=0.28	BSD	ASGI server for FastAPI

3.2 Frontend

Technology	Version	License	Purpose
React	v19	MIT	Component-based UI framework
Vite	v6.4	MIT	Ultra-fast dev server and production build tool
Tailwind CSS	v4	MIT	Utility-first CSS — all styling via class names
Chart.js	v4.4	MIT	Canvas-based interactive charting engine
React-ChartJS-2	v5.3	MIT	React wrapper for Chart.js components
React Router	v7	MIT	Client-side SPA routing (no page reloads)
Axios	v1.7	MIT	HTTP client for API communication
Lucide React	v0.475	ISC	Professional SVG icon library

3.3 Database

Database	Use Case	License
SQLite	Local development — zero config, file-based (facilityops.db)	Public Domain
PostgreSQL	Production — enterprise RDBMS via DATABASE_URL env variable	PostgreSQL License

4. System Architecture

The platform follows a clean **three-tier architecture**. The Maintenance module introduces a new set of Agents and Services that work alongside the Energy Module, sharing the same DB.

```
PRESENTATION LAYER (React 19 + Vite + Tailwind CSS)
PM Dashboard | Equipment | Health Scores | Predictions | Schedule | PM Alerts
| HTTP REST (Axios)
APPLICATION LAYER (FastAPI Python)
API Routers: /maintenance/* /equipment/*
| |
AI Maintenance Agent Maintenance Service
Predictive Rule Engine Health Scoring Logic
Anomaly Detector Schedule Generator
| SQLAlchemy ORM
DATA LAYER (SQLite / PostgreSQL)
Tables: equipment | maintenance_monitoring | maintenance_alerts | schedule
^
IoT DATA SOURCE LAYER
Microsoft Azure Predictive Maintenance Dataset (PdM CSVs)
```

5. Database Design

Table 1: equipment (100 machines from Azure dataset)

Column	Type	Constraint	Description
equipment_id	INTEGER	PRIMARY KEY	Matches machineID (1-100)
facility_id	VARCHAR(50)	FK	Facility mapping
equipment_name	VARCHAR(150)	NOT NULL	Name of asset
equipment_type	VARCHAR(100)	NOT NULL	model1..model4
status	VARCHAR(50)	DEFAULT Operational	Operational / Under Maintenance
expected_life_years	INTEGER	—	Mapped from Azure 'age' field

Table 2: maintenance_monitoring (~72,000+ IoT telemetry rows)

Column	Type	Description
monitoring_id	INTEGER (PK)	Auto-incrementing primary key
equipment_id	INTEGER FK	Foreign key → equipment.equipment_id
timestamp	DATETIME	Hourly IoT sensor reading timestamp
power_consumption	FLOAT	Mapped from Azure 'volt'
vibration	FLOAT	Mapped from Azure 'vibration'
pressure	FLOAT	Mapped from Azure 'pressure'
runtime_hours	FLOAT	Cumulative runtime
operating_status	VARCHAR(50)	Running Idle Fault

Table 3: maintenance_alerts & maintenance_schedule

Table	Key Columns	Description
maintenance_alerts	alert_id, equipment_id, issue, severity, status	Populated by Azure PdM_errors.csv (Medium severity) and PdM_failures.csv (Critical)
maintenance_schedule	schedule_id, next_service_date, maintenance_type, status	Populated by Azure PdM_maint.csv (Component replacements)

6. Backend — FastAPI Application

The backend auto-seeds the predictive maintenance data on startup by running the `generate_pdm_dataset.py` generator or downloading the Azure CSVs directly, injecting them into the DB.

Folder Structure (Maintenance Extensions)

```
backend/  
app/  
models/  
equipment.py Equipment, MaintenanceMonitoring, Alerts, Schedules  
api/  
maintenance.py GET /api/maintenance/dashboard, /api/equipment  
services/  
maintenance_seed_service.py Loads PdM CSVs into the ORM on startup  
replay_service.py Background daemon simulating live IoT traffic  
data/pdm/  
PdM_machines.csv  
PdM_telemetry.csv (72,000 hourly rows)  
PdM_errors.csv  
PdM_failures.csv  
PdM_maint.csv  
generate_pdm_dataset.py Generates faithful Azure PdM dataset offline
```

7. AI Maintenance Agent — Health & Predictions

The **AI Maintenance Agent** continuously evaluates telemetry streams to assign Health Scores and predict failure probability.

7.1 Health Score Algorithm

Each machine starts with a base score of 100. The agent subtracts points based on:

- **1.** Active Critical Alerts: -20 points per alert.
- **2.** Active High/Medium Alerts: -10 to -5 points per alert.
- **3.** Overdue Maintenance: -15 points if a scheduled service is missed.
- **4.** Vibration/Pressure Spikes: -2 to -5 points for recent telemetry deviations.

7.2 Predictive Inference

Trigger Condition	Inference Output	Recommended Action
Vibration > 2 Std Devs	Bearing wear detected	Inspect motor bearings immediately
Pressure Drop + Volt Spike	Pump cavitation / Seal leak	Schedule pump maintenance
Repeated Error1 + Error2	High breakdown probability (85%)	Proactive component replacement
Age > 15 yrs + High Load	End of lifecycle approaching	Budget for capital replacement

8. Analytics & Scoring Engine

The Pandas-powered analytics engine aggregates the vast telemetry data into human-readable KPIs.

Metric	Calculation	Used In
Overall Equipment Health	Average of all equipment health scores	PM Dashboard KPI
Machines at Risk	Count where health_score < 70	PM Dashboard KPI
Active PM Alerts	Count of unresolved alerts in maintenance_alerts	PM Dashboard KPI
Upcoming Schedules	Count of maintenance_schedule in next 7 days	PM Dashboard KPI
Historical Health Trend	Daily average health score over 30 days	Line Chart
Downtime Risk Distribution	Bucket machines into Low/Med/High risk	Doughnut Chart
Telemetry Timeseries	Plot 24h pressure/vibration/volt traces	Equipment Details

9. REST API Endpoints

All endpoints at **http://localhost:8000**.

Method	Endpoint	Description
GET	/api/maintenance/dashboard	Aggregated stats: Health avg, at-risk count, active alerts
GET	/api/equipment	List all equipment, filter by facility_id
GET	/api/equipment/{id}	Get specific equipment details + 30-day telemetry
GET	/api/maintenance/alerts	Fetch active predictive alerts
PATCH	/api/maintenance/alerts/{id}	Acknowledge or Resolve an alert
GET	/api/maintenance/schedule	Fetch upcoming preventive/corrective schedules

10. Frontend — React Dashboard

The frontend uses an accordion-style collapsible sidebar to cleanly separate the **Energy Agent** and **Maintenance Agent** modules.

Reusable UI Components

Component	File	Purpose
	layout/Sidebar.jsx	2-row accordion layout, auto-expands based on route
	maintenance/EquipmentCard.jsx	Displays machine info, health score radial, and status badge
	maintenance/HealthGauge.jsx	Circular progress bar (Green/Yellow/Red) for health score
	maintenance/AlertList.jsx	Interactive list to manage and acknowledge PM alerts

11. Dashboard Pages & Features

/maintenance/pm Dashboard

Executive overview for maintenance. KPIs: Avg Health, Machines at Risk, Active Alerts. Charts: 30-day Health Trend, Risk Distribution, Facility-wise comparison.

/maintenance/equipment Equipment

Inventory view of all machines. Displays age, type, location, and real-time health score. Clicking a machine opens detailed telemetry traces.

/maintenance/health Health Scores

Deep dive into the health scoring algorithm. Shows leaderboards for most healthy and least healthy assets. Provides AI reasoning for point deductions.

/maintenance/predictions Predictions

AI inference hub. Displays equipment with high probability of failure in the next 7-30 days based on telemetry anomalies.

/maintenance/schedule Schedule

Calendar and list view of upcoming maintenance tasks. Filter by Preventive, Corrective, or Predictive type.

/maintenance/alerts PM Alerts

Inbox for AI-generated warnings (Vibration Spikes, Pressure Drops, Error Codes). Engineers can update status to In Progress or Resolved.

12. Data Simulation & Microsoft Azure Dataset

Instead of random data, this module leverages the real-world **Microsoft Azure Predictive Maintenance** dataset from Kaggle. A local script replicates the exact schema and statistical distributions of the dataset, providing highly realistic telemetry.

File	Rows	Description
PdM_machines.csv	100	100 machines (model1..model4) with ages
PdM_telemetry.csv	72,000+	Hourly voltage, rotation, pressure, vibration
PdM_errors.csv	~3,919	Non-failure error events logged during operation
PdM_failures.csv	~761	Component failure events (critical)
PdM_maint.csv	~3,286	Component replacement/maintenance records

Live Replay Service: A background daemon (`replay_service.py`) reads the CSVs and inserts rows every 15 seconds to simulate real-time live IoT streaming for demos.

13. Report Export Engine

Format	Library Used	Content
CSV	Python csv module	Raw equipment list or maintenance logs
Excel (.xlsx)	OpenPyXL >= 3.1	Formatted health score reports and upcoming schedules
PDF	ReportLab >= 4.1	Formal maintenance audit document with downtime stats

14. Licenses & Open Source

All tools, frameworks, and libraries are 100% free and open-source (MIT / BSD / PostgreSQL licenses). The Azure Predictive Maintenance dataset is provided openly for ML experimentation.

15. How to Run the Application

Prerequisites

- Python 3.11+
- Node.js 18+ and npm

Start Backend (FastAPI)

```
cd backend
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

The backend will auto-generate the Azure PdM dataset on first boot (takes ~5 seconds) and seed the DB.

Start Frontend (React + Vite)

```
cd frontend
npm run dev
```

Dashboard available at: <http://localhost:5173>

Demo Mode / Live IoT Streaming

Enable `REPLAY_MODE` in `backend/.env` to simulate live telemetry inserts every 15 seconds.

16. Future Roadmap

Module	Description	Status
Energy Intelligence & Monitoring	Real-time IoT telemetry, AI anomaly detection	ACTIVE
Predictive Maintenance	ML-based equipment failure prediction, health scores	ACTIVE
Occupancy & Space Optimization	People-count analytics, zone utilisation	Upcoming
Security & Anomaly Intelligence	Access control monitoring, CCTV event classification	Upcoming
Cost Intelligence & Billing	Tariff analysis, demand charge optimisation	Upcoming

End of Documentation — Agentic FacilityOps AI Platform v1.0 | Predictive Maintenance Module

Document generated: 15 August 2026, 10:05 PM